

Ideate v3 — Architecture Inventory & Usage Audit

Ideate v3 — architecture inventory, part 0: overview

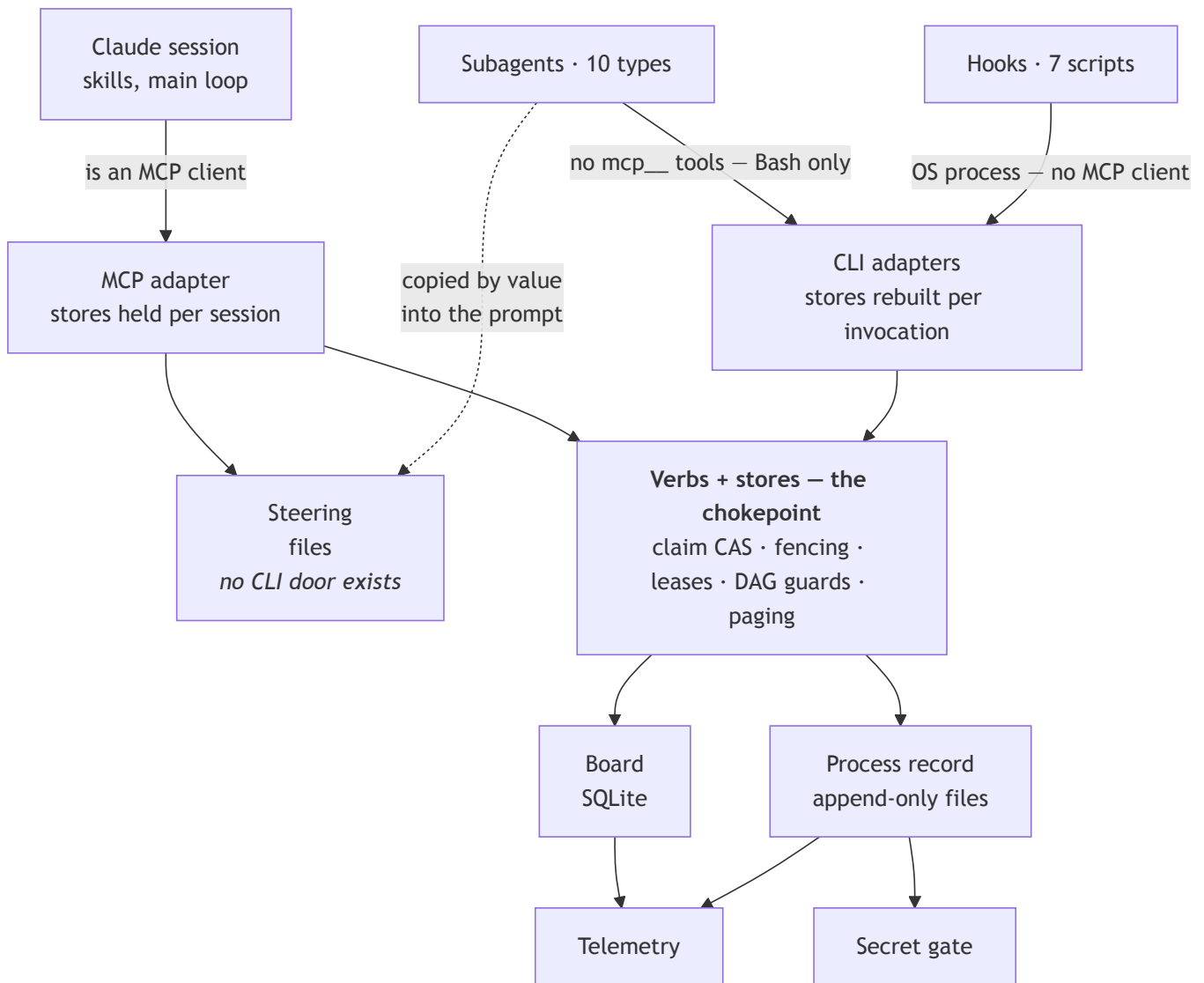
What the shipped MVP actually is, stated from the code (cross-module import census + reading each crossing), 2026-08-03. This set of docs is the input to the alignment session that decides what the stage-3 eval harness measures.

The one-paragraph version

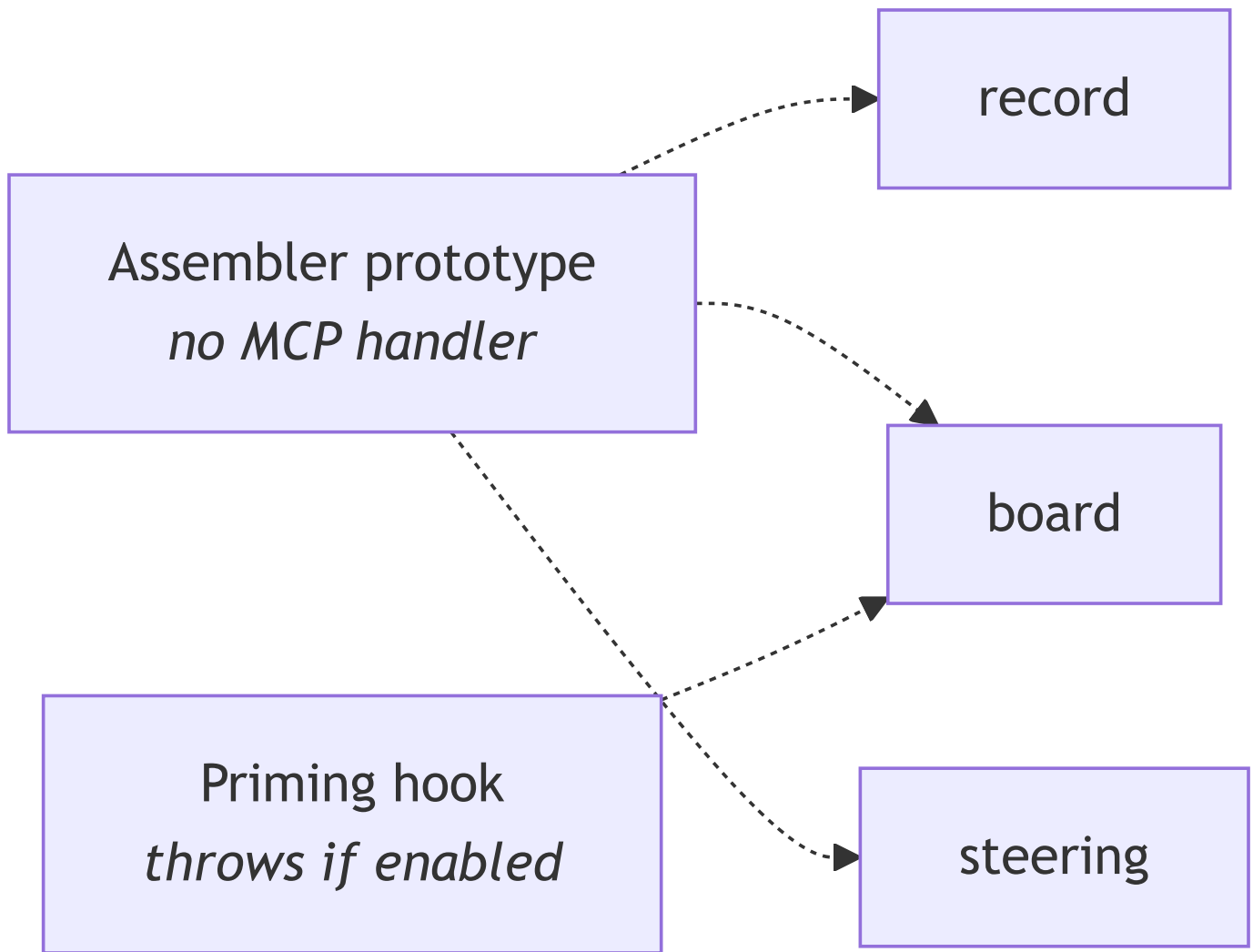
Ideate v3 is a Claude Code plugin that gives an agentic project a **memory it can converge against**: an append-only **process record** (why things were decided), a **delegation board** (what work exists, who holds it, what's done), and a **steering store** (the principles and policies that govern the work). Callers reach the stores through an MCP server (one process per Claude session) or per-invocation CLIs, over the same on-disk files — not a choice between equals, but because hooks are OS processes and subagents are granted no MCP tools, so the CLI is the only door either one can speak. A third function from the ratified frame, the **knowledge graph**, is scheduled post-MVP and exists today only as the record store's typed references and a gated-off prototype assembler (GP-23: no intelligence machinery before the instrument that measures it).

Component map

The doors are **not** alternative routes for the same caller. Each caller has exactly one it is *capable* of speaking, and the shape of the diagram is a consequence of the host, not a design preference:



The gated layer sits beside all of this, wired to nothing (GP-23):



Three access patterns, not two: MCP for the session, the CLIs for every caller that runs as a *process* rather than inside the session, and — for steering, which has no CLI door at all — **copy-by-prompt**, where the invoking skill reads steering over MCP and pastes it into the subagent’s prompt (agents/domain-curator.md). The third pattern is real, load-bearing, and was invisible in earlier versions of this map.

The three stores, one line each

Store	On disk	Mutation model	Why it’s separate
Process record	Markdown files, one per record	Append-only; corrections are new records that <i>supersede</i>	The audit trail. Files = no version handshake = survives outages (conway)
Delegation board	board.db (SQLite)	Highly mutable, transactional: claim	Coordination state. Compare-and-set needs a real database

Store	On disk	Mutation model	Why it's separate
		CAS, fencing tokens, leases	
Steering	Files (YAML-frontmatter items)	Mutable, MCP-only writes, amendment history kept	Governance cadence: amended rarely, read every session

What “thin seam” means here, mechanically

A census of the shipped source’s cross-module imports shows the coupling is exactly as narrow as the frame claims:

- The stores **never import each other** — with two named exceptions, both the same *shape*: a small purpose-built bridge appending one kind of board event to the process record (`completion-record.ts` , `migration-signal.ts`). This is the named **bridge pattern**: board events that matter to the process record cross via a bridge module, never via the stores knowing each other.
- One module, `transport/id-resolver.ts` , is allowed to know *both* the record and the board — it answers “which store does this id belong to.” That is its entire job.
- Everything else that looks like a cross-seam import is `record/id.js` (the `Clock` type and ULID generator) — a shared kernel living inside `record/` by accident of history. Named here so future censuses don’t re-explain it.

The transports are a designed fault line

The CLI builds a fresh store per invocation; the MCP server holds one for the session. Any behavior whose mechanism depends on process lifetime reaches exactly one of them — the split produced four defects in one arc before it got an explicit contract (`docs/transport-contract.md`): a two-question invariant (staleness of cross-call state; discovery across both transports’ consumers) plus a cross-transport parity test.

The chokepoint that limits the blast radius is **one layer below the transports**, and it is real: both adapters do the same three lines — `loadConfig` → `new WorkStateStore` → `new WorkStateVerbs` — and then call `ctx.verbs.<verb>` . No claim CAS, fencing rule, lease policy, or DAG guard is written twice (`cli/ideate-work.ts:243` , `work-state/tools.ts:335`).

What is *not* enforced is the adapters’ thinness. Each one independently decides which optional seams to wire, and they have already diverged — see `04-transports-and-infra.md` for the two verified cases. The fault line is governed; the **surface** is not.

The mechanical-enforcement layer is part of the architecture

Ideate's conventions are held by *tests that scan the shipped tree*, not by discipline: census tests derive their registries by scanning `skills/`, `agents/`, `docs/` at test time (a new prose surface auto-enters and fails until it complies or carries a reasoned exemption); parity tests run the real CLI binary against real stores; mutation-testing is the stated standard (P-41) for any guard. This layer is why “lightweight docs” can stay lightweight — the load-bearing invariants live in executable checks, and these docs point at them rather than paraphrase them.

Where the rest of this set goes

- `01-process-record.md` — how the record stores and serves data
- `02-delegation-board.md` — the board's storage, claim lifecycle, migrations
- `03-steering.md` — the steering store and its consumption
- `04-transport-and-infra.md` — the two doors, hooks, telemetry, guards
- `05-usage-audit.md` — **the audit**: what each data area is *for*, who actually reads it, what's working, what isn't, and the improvement list — the direct input to the alignment session. # Ideate v3 — architecture inventory, part 1: the process record

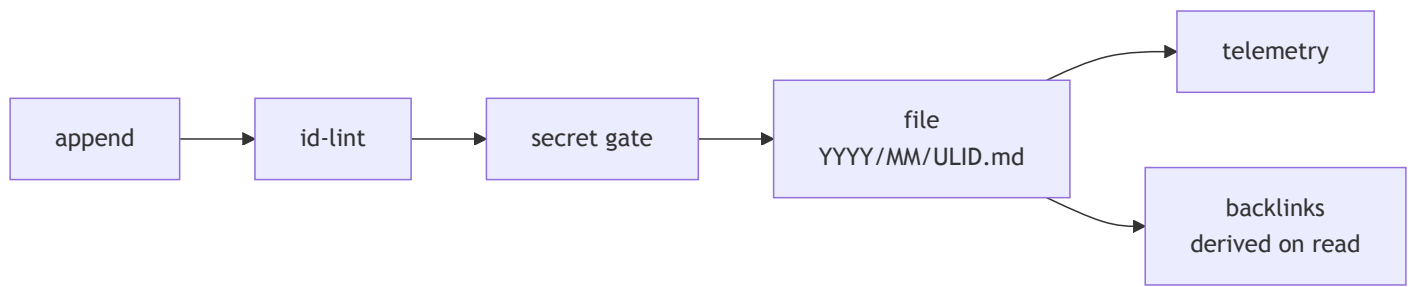
The append-only store of why: decisions, findings, completions, journal entries. Source of truth: `src/record/` (`store.ts`, `read-page.ts`, `tools.ts`), `src/cli/ideate-record.ts`, `src/cli/record-walk-snapshot.ts`.

Storage

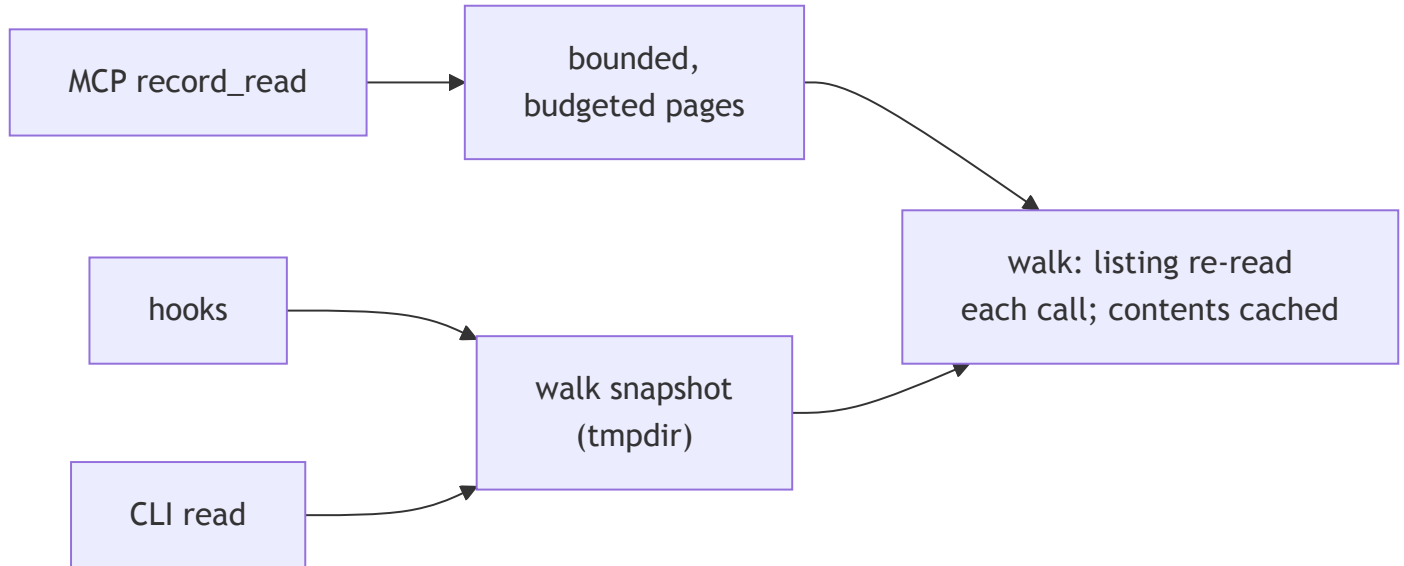
One Markdown file per record — YAML frontmatter (id, kind, claim, scope, source, references) plus a prose body — with a ULID filename stem, date-sharded as `record.path/YYYY/MM/{id}.md`. The record path comes from `.ideate.json` (default `.ideate/record/`).

Three properties do the load-bearing work:

- **Append-only, immutable.** No code path edits a record. A correction is a *new* record carrying a `supersedes` edge; the superseded record surfaces its replacement as a derived backlink on read. This is what makes the permanent content cache safe (below) and what made records survive the conway board outage untouched.
- **Files, not a database.** No version handshake, no migration, openable by anything that can read Markdown — including a human with `grep`. Layout is frozen by steering (GP-20).
- **Secret-gated writes.** Every append passes a secret-scanning gate (`secret-gate/`) before persisting; redactions are counted in telemetry.



Retrieval — three shapes, one walk



- **Selection, never ranking.** `record_read` filters (by id, or a plain substring over scope/kind/source) and pages; it does not score. Ranking is the future assembler's job, not the store's.
- **Bounded by two budgets.** Every list page is capped by count *and* by a payload character budget, with keyset cursors — the fix for the 81KB single-line `work_list` overflow class.
- **Freshness is structural.** The directory listing is re-read on every call, so a record written by any other process (the CLI, another session) is visible on the very next read; only immutable file contents are cached. Cross-transport visibility is pinned by `transport-parity.test.ts`.
- **The CLI's O(N) walk problem** (a fresh store per invocation re-walked every record on every call) is solved by an ephemeral per-project snapshot under `os.tmpdir()`, keyed by store path + selection + a freshness token — a 2.00× measured speedup with a byte-identical envelope, falling back to a live bounded page on any staleness.

Who actually reads it

Consumer	Path	Shape
Session-start hook	CLI	10 most recent records as a context digest
Subagent-start hook	CLI	same digest, delivered to every spawned agent
Skills (execute/review/refine/autopilot)	MCP <code>record_read</code> + CLI	scope-filtered and by-id reads
Pre-compact / session-end / commit / task hooks	CLI	writes — the capture floor

Scale today

The two dogfooding stores on this machine: the ideate repo’s own store (~3,100 records) and the plugin dev store (~1,800). Reads stay paged and budgeted at this scale; the CLI snapshot keeps per-invocation cost flat.

Known edges

- **No semantic search.** Retrieval is recency + substring selection only. The hybrid semantic-seed + PPR direction is parked, deliberately, behind GP-23’s instrument.
- **kind is open vocabulary.** Disciplined by convention (decision, finding, work-completion, journal, ...), not schema — cheap to write, and the reason “board-migration” could become a kind with zero code change. # Ideate v3 — architecture inventory, part 2: the delegation board

The coordination state: what work exists, its dependencies, who holds it, what’s done. Source of truth: `src/work-state/` (`schema.ts`, `store.ts`, `claims.ts`, `expiry.ts`, `verbs.ts`, `tx.ts`, `dag.ts`, `tools.ts`).

Storage

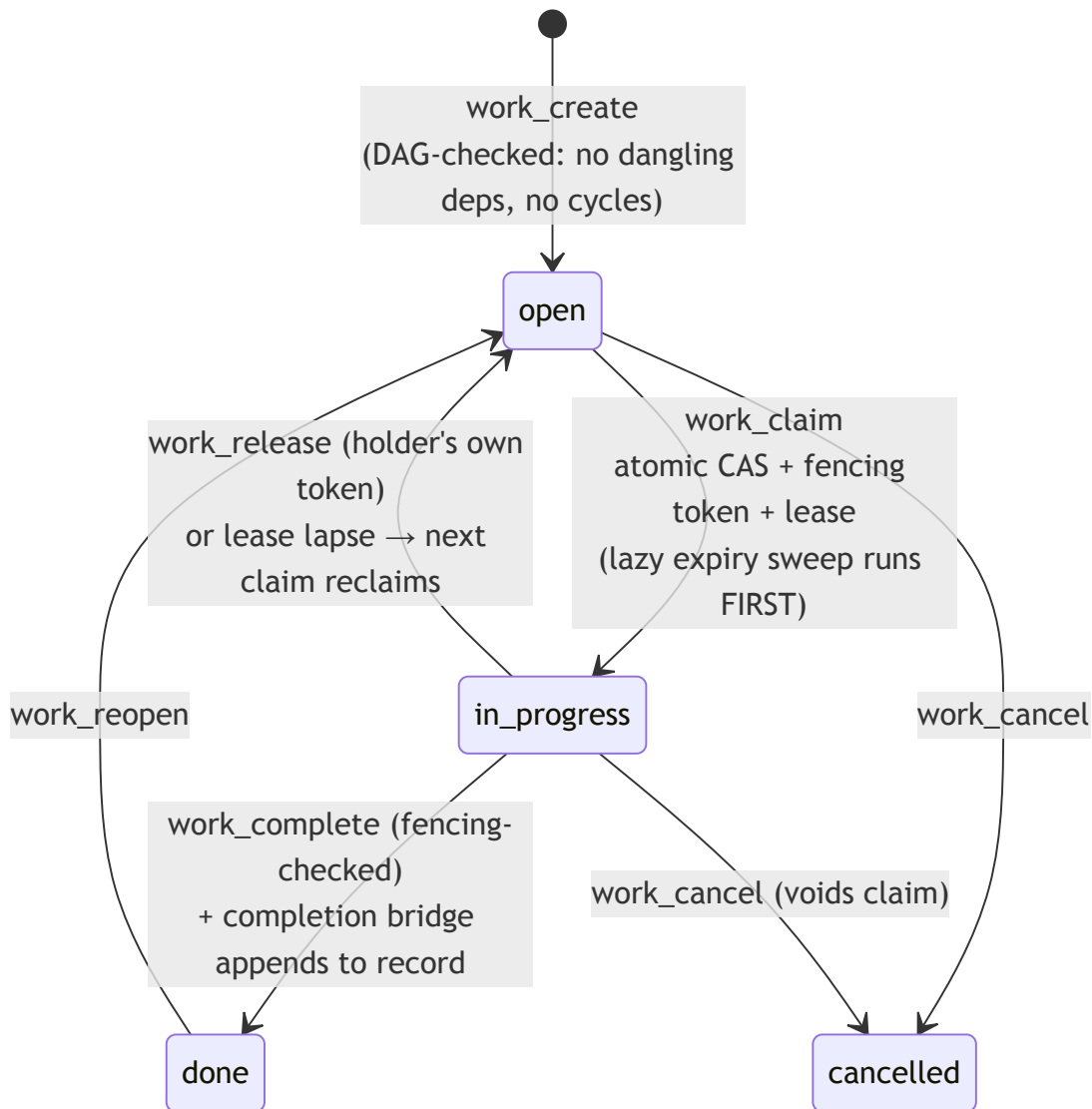
A single SQLite file (`board.db` under `.ideate-work/`), WAL mode with a busy-timeout set on every connection by construction — two simultaneous sessions writing one board is ordinary, not exceptional. Two tables:

- **items** — one row per work item: title, spec, status, `depends_on` (JSON), `parent_id` (containment), `"references"` (typed forward edges, e.g. supersedes), actor stamps, a version counter for optimistic CAS, and the five claim columns (holder, fencing token, lease).

- **events** — the append-only transition log. No code path may UPDATE or DELETE it (grep-falsifiable; pinned by test). Every claim, release, completion, cancellation, and expiry-reclaim lands here with actor and timestamp — the board's own audit trail.

Lazy-init: nothing on disk until the first write; reads against a never-written board return empty without creating anything.

The claim lifecycle — where the correctness lives



Three mechanisms carry the multi-agent guarantees:

1. **Fencing tokens.** Every claim mints a monotonically increasing token from a counter column (not derived from events, so it survives claim deletion). `complete / renew / release` reject a stale token — an agent whose lease lapsed and was reclaimed cannot clobber the new holder.
2. **Leases with lazy expiry.** Claims expire (default 4h); the expiry sweep runs lazily at the start of every id-scoped claim verb, as its own atomic transaction, and logs the reclaim to `events`.

3. **Optimistic CAS on metadata.** `work_update_meta` carries an expected version; dependency edits are DAG-checked (no dangling refs, no cycles, no self-ancestry) at write time.

Schema versioning — the one-way door, now with a window

`PRAGMA user_version` stamps the schema version (currently 3); `PRAGMA application_id` stamps a **compatibility floor** — the oldest binary verified safe against the current schema. A newer board whose floor covers this binary opens *degraded with a loud warning* instead of hard-refusing; a board with no floor or a floor above this binary still throws `SCHEMA_VERSION` — the refusal stays loud by design. Every migration announces on stderr at the moment it runs **and** fires a listener that appends a durable `board-migration` record to the project's process record (the conway incident had nothing pointing back at the crossing; now it would). Boards written before the floor existed self-heal the stamp on their next write.

Bridges to the process record (the named pattern)

Board events that matter to the process record cross via small purpose-built bridge modules — the stores never import each other:

- `completion-record.ts` — `work_complete` appends a `work-completion` record (best-effort: record failure never blocks the completion, and is loud on stderr + telemetry).
- `migration-signal.ts` — a schema migration appends a `board-migration` record through a process-scoped listener registered by each transport.

Retrieval

List reads are **selection, never ranking**: filter by tenant/status/parent, page to exhaustion with `next_cursor` until it is `null`, bounded by both a count limit and a payload budget (same `transport/` machinery as the record store). Claimability is computed over the *whole* board, not the page, so paging can never hide a claimable item. Every read opens a fresh connection — there is no cross-call cache to go stale, which is why the board was immune to the record store's transport-split correctness regression.

Who actually uses it

The five skills (execute/review/refine/autopilot/init) drive it via MCP tools; agent prompts and hooks reach it via the `ideate-work` CLI. The read-side census of every prose surface that mentions the board read is mechanically pinned (`board-paging-vocabulary.test.ts`) — the registry is derived by scanning the shipped tree, so a new consumer auto-enters.

Scale today

The ideate repo's own board: ~160 items, all but one done at the time of writing; the events log spans the full history of every transition since the board's creation. # Ideate v3 — architecture inventory, part 3: the steering store

The governance layer: guiding principles (GP-) and policies (P-) that the work is steered by. Source of truth: `src/steering/` (`store.ts`, `tools.ts`).

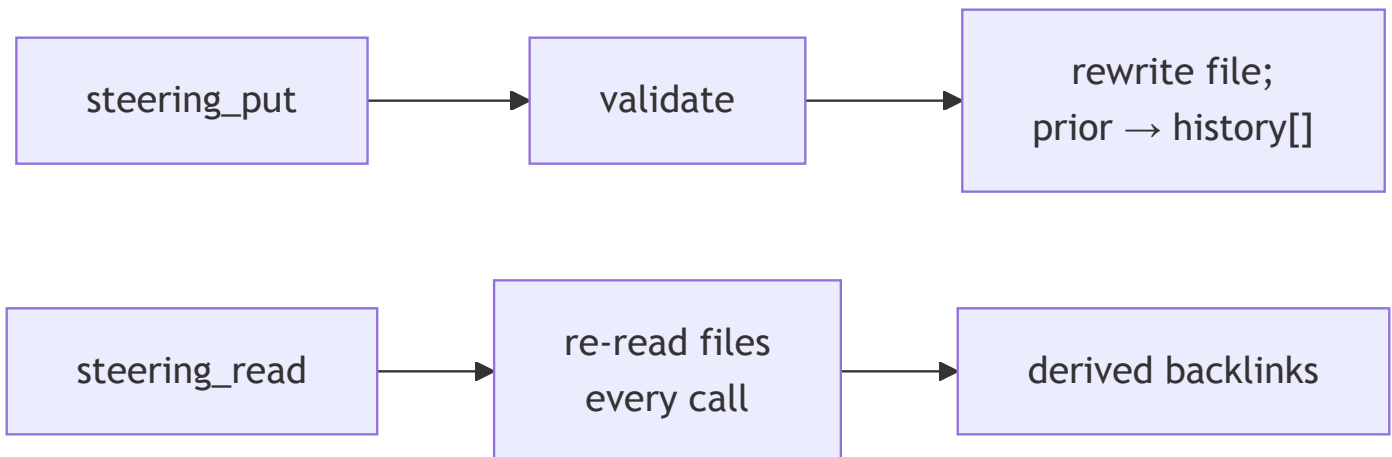
Why it exists in the frame

The ratified frame names three functions — process record, knowledge graph, delegation board. Steering is the fourth, shipped and fundamental: it is where the project's *rules about the work* live (eval-first, narrow seams, mutation-proven tests, human-presentation), distinct from the record of what happened and the board of what's being done. The frame's naming should say “three functions + steering” — this store is not an implementation detail.

Storage

One Markdown file per item under `.ideate/steering/` — quoted-YAML frontmatter (id, kind, domain, status, amendment `history`) plus the statement as prose body. ~105 items on this machine's ideate store (active + deprecated + superseded; the active set is smaller).

Deliberately primitive: **read straight off the files — no index, no cache.** Steering items are *mutable* (amendments are the whole point), so the record store's “cache contents forever” strategy does not transfer — a steering contents-cache would go stale the moment any process amends an item. Each amendment appends the prior version to the item's `history`, so the amendment trail is inside the file itself.



Two properties worth naming

- **MCP-only writes.** There is no `ideate-steering` CLI bin — a deliberate single-transport write path, so the two-transport fault line (see `docs/transport-contract.md`) does not apply to writes at all. Reads happen through MCP tools and the in-process assembler path.
- **Gated per project** (`steering.enabled` in `.ideate.json`). The store tools are gated OFF by default (GP-23’s naming discipline); the v2→v3 migration enabled steering in all 18 migrated projects.

Who actually reads it

Consumer	How
refine / review / autopilot / init skills	<code>steering_read</code> (policies shape planning, review severity, curation)
domain-curator agent	proposes amendments; the invoking skill applies them via <code>steering_put</code>
review’s steering-curation pass	<code>steering_put</code> amendments with history
This architecture review	GP-23, GP-26, P-40, P-41... are steering items, not folklore

The closed loop it’s part of

Steering is how a *lesson* becomes a *rule with an audit trail*: an incident (conway’s lockout) → a finding → a decision → a policy amendment (P-40’s transport-parity clause) whose full history is in the file, greppable and citable. The census/guard tests then cite the steering item they enforce (P-48, P-52, P-41 appear verbatim in test headers) — so a principle’s *mechanism*, when one exists, is traceable from the item itself. GP-23’s mechanism is precisely what stage 3 exists to restore. # Ideate v3 — architecture inventory, part 4: transports, hooks, and infrastructure

Everything between the host and the stores, and everything beside them. Source of truth:
`src/cli/` , `src/server.ts` , `src/transport/` , `src/telemetry/` , `src/secret-gate/` , `src/config/` , `hooks/` .

The two transports (one fault line, now governed)

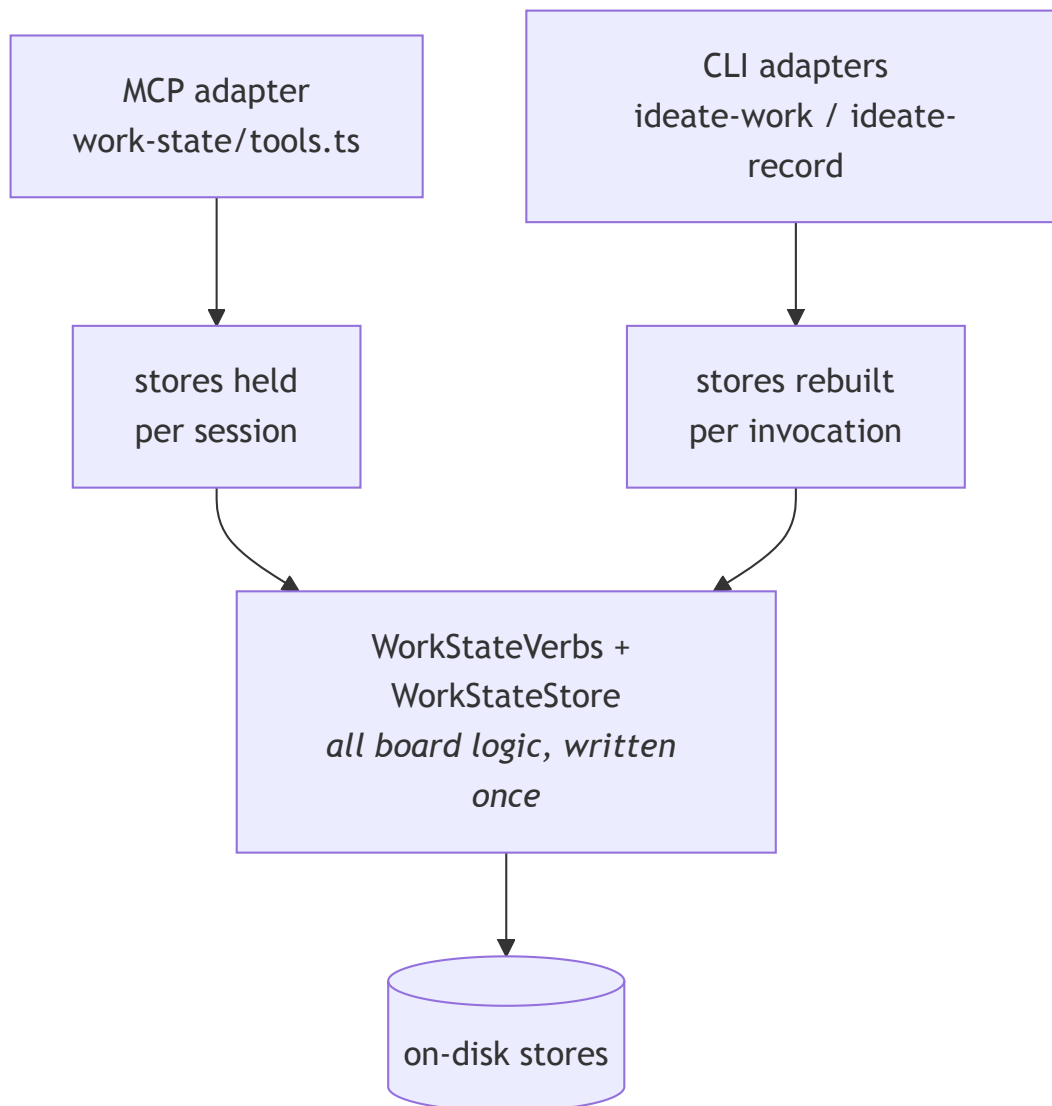
Why there are two doors at all

Not redundancy, and not a choice we can revisit by deleting one. The two doors exist because the callers are not all the same kind of thing:

Caller	Can it speak MCP?	Why
The Claude session (skills, main loop)	Yes	It <i>is</i> the MCP client
Hooks (<code>hooks/*.mjs</code>)	No	They are OS processes the host executes and reads stdout from. There is no MCP client in a hook
Subagents (all 10 in <code>agents/</code>)	No	Every agent's frontmatter grants <code>Read</code> , <code>Grep</code> , <code>Glob</code> , <code>Bash</code> (+ <code>Edit/Write</code> for worker, <code>WebSearch/WebFetch</code> for researcher). Not one grants an <code>mcp__</code> tool — an MCP connection belongs to the session and is not inherited by a spawned subagent

So the CLI is the *only* door for two thirds of the caller population. Six of the ten agents name the CLIs explicitly in their prompts. Removing the CLI would not simplify the architecture; it would disconnect the hooks and every subagent from the board and the record.

Adapters over a shared core



Same stores, same envelopes, same budgets — different process lifetimes. That difference is the fault line documented in `docs/transport-contract.md` (two-question invariant: staleness of cross-call state; discovery across both transports' consumers), guarded by `transport-parity.test.ts` (a real CLI child process against a warm MCP-style store, both directions) and steering rule P-40's transport-sibling clause.

The surface has already drifted (two verified cases)

`cli/ideate-work.ts`'s own header claims "the eleven-verb surface, mirrored here one subcommand per verb." Read against `work-state/tools.ts`, that is not currently true. The *core* is shared; the *adapters* each decide independently which arguments and optional seams to wire, and nothing checks that they made the same decisions.

1. **Containment edges are unreachable from the CLI.** `parent_id` appears **zero times** in `cli/ideate-work.ts`. The MCP surface accepts it on both `work_create` and `work_update_meta` (set / move / clear-to-root, with the absent-vs-present-null distinction carefully preserved). Consequence: a subagent — which has no other door — cannot create or change the hierarchical containment edge, one of the board's two edge types and the thing `ideate:decomposer` exists to produce.
2. **Dangling reference ids fail silently on the CLI.** `verbs.updateMeta` takes an optional `onUnresolvedIds` callback (`work-state/verbs.ts:588`). MCP passes it at three sites — `work_create`, `work_update_meta`, `work_release` — and returns `unresolved_ids` in the envelope. The CLI passes it at none, so a typo'd `--supersedes <id>` is accepted with no warning on the one door subagents have. The id-lint that surfaces those ids was itself added by a correction (`work-state/tools.ts:417` cites finding `01KYV387QKRP3V330WAS6DX95K`) — and the fix landed on one door only.
3. **The CLI silently forks a new store when `cwd` is not the project root.**
`loadConfig(projectRoot)` reads `<projectRoot>/.ideate.json` and, on `ENOENT`, *lazily creates it* (`config/ideate-config.ts:134`). There is no walk-up to find an enclosing project. The CLI passes `process.cwd()` (`cli/ideate-work.ts:324`), so running `ideate-record` from a subdirectory does not attach to the project's store — it silently onboards a **brand-new empty one** there. The MCP server has the same default (`work-state/tools.ts:338`) but is immune in practice: the host sets its `cwd` to the project dir once, at session start, and it never changes. The hooks are immune too, and deliberately — they pass `cwd: projectRoot` explicitly (`hooks/session-start.mjs:42`). Subagents are the exposed caller: nothing pins their working directory before a `Bash` CLI call.

This is not hypothetical. Compiling *this document set* left a phantom store in `docs/architecture/` — `.ideate.json`, `.ideate-telemetry/`, and nine records under `.ideate/record/2026/08/` — written by census CLI runs that happened to have that directory as their `cwd`. Those nine records are invisible to the project they were meant for.

Neither of the first two is caught today, because `transport-parity.test.ts` is three tests about **record-store freshness** (`src/record/transport-parity.test.ts:101`). It proves writes cross the process boundary. It says nothing about the board, and nothing about whether the two doors expose the same verb surface.

Steering has no CLI door — it is copied by prompt

There is no `ideate-steering` binary; `src/cli/` holds `record`, `work`, and a snapshot utility, and no CLI file mentions steering. Steering reaches subagents by **value**: the invoking skill reads it over MCP and pastes it into the prompt (`agents/domain-curator.md` says so explicitly — “steering is MCP-only for writes, but the skill hands you...”). That is a deliberate and defensible choice for a

store amended rarely and read every session, but it is a third access pattern with its own costs (prompt bulk, and a snapshot that cannot refresh mid-agent), and it belongs on the map rather than in the gaps between the other two.

Evaluated as two layers (per the review’s tech-debt finding): `src/transport/` is (a) shared read-path utilities — payload budget, keyset paging, id-lint — a *lower* layer both stores legitimately use; and (b) `id-resolver.ts`, the one module allowed to know *both* the record and the board — a *composition*-layer citizen. Both layers are working as intended today; the naming doesn’t say there are two, which is the debt.

Hooks — the zero-cooperation capture floor

Seven host lifecycle hooks (`hooks/` , wired by `hooks.json`) make capture happen without any skill remembering to do it:

Hook	What it does
SessionStart	primes the session with the 10-most-recent record digest (CLI) + opportunistic board sweep
SubagentStart	delivers the same digest to every spawned subagent — primes <i>other frameworks’</i> workers too
PreCompact	captures session knowledge at the moment compaction would destroy it
SessionEnd	session-outcome record + board sweep
PostToolUse (git commit only)	commit-boundary record — the one completion signal every workflow emits
SubagentStop	delegated-work outcome records for any framework’s subagents
TaskCompleted	completion analogue for Claude Code’s native task list

Design stance: hooks are the **floor** (always-on, bounded, cheap), skills are the **rich path** (structured, deliberate). The digest is 10 records by design — high-signal-minimal, not a firehose.

Telemetry — collected, mostly unconsumed (audit flag)

Append-only NDJSON counters per project (`.ideate-telemetry/`). Seven counter names exist; four have writers:

Counter	Written by	Read by
<code>capture_fired</code> , <code>capture_write_failed</code> , <code>redactions</code>	record store, completion bridge	tests + the <code>ideate-telemetry</code> diagnostic bin
<code>work_claims</code>	priming hook (every claim — the future eval’s denominator)	the report builder
<code>priming</code> , <code>kg_unreachable</code> , <code>frontier_size</code>	nothing — zero increment call sites	the report parser only

Nothing automated consumes the report: no hook, skill, agent, or doc calls `ideate-telemetry`. This is the audit’s clearest “data we have but don’t use” finding — kept deliberately cheap to collect, waiting on the stage-3 harness to say which signals matter. The three zero-caller counters are leftovers of the retired usage surface / KG design and should be re-specified from the new harness’s requirements, not resurrected.

The gated layer (GP-23’s gate, held)

- `context/assemble-prototype.ts` — the budgeted, provenance-bearing briefing composer (structure-first seeding from board edges, density packing, per-source caps). Proves the three stores compose read-only. **No MCP handler, no wiring** — gated until the eval that measures it exists.
- `work-state/priming-hook.ts` — the seam in the claim path where priming will run. Flag off by default; flipping it throws `NOT_IMPLEMENTED` (caught, logged, swallowed — a claim can never fail because the flag was flipped early). Its only live effect is the `work_claims` counter.

Config

`.ideate.json` per project (schema_versioned, honest-failure on a newer file): record path, backend (`local` today, hosted later), work-state path, steering enablement. `loadConfig` lazily onboards — first call creates the file and record directory. The plugin’s runtime floor is Node $\geq 22.5.0$ (`node:sqlite`), pinned in `package.json` engines. # Ideate v3 — architecture inventory, part 5: the usage audit

The point of the inventory: each data area, what it’s for, who actually reads it, and an honest verdict. Evidence-linked throughout; this is the direct input to the alignment session. Compiled 2026-08-03 against the shipped tree and the two dogfooding projects (ideate + plugin dev).

Per-area audit

Data area	What it's for	Who reads it	Verdict
Process record (~3.1k records, ideate repo)	The why: decisions, findings, completions, journal	Session/subagent digests (10 recent), skill MCP reads, by-id fetches, scope filters	Working, but the capture is polluted. The conway handoff record proves the value: work survived a dead board <i>because</i> the record was there. What arrives is another matter — see “what’s not working” item 6
Board items (~160 on ideate)	Work queue + decomposition	Every skill, every session	Working. The convergence oracle — “zero open claimable work” is the stage-completion signal
Board events log	The board’s own audit trail	<code>work_events</code> reads (rare; debugging/handoffs)	Working, under-read. Rich history, consumed mostly at incident time
Steering items (~105)	The rules of the work	refine/review/autopilot reads; curator amendments	Working. The incident→finding→decision→amendment loop is real (P-40’s transport clause)
Telemetry counters	Capture-funnel + future eval signals	Nobody automated; manual <code>ideate-telemetry</code> only	Collected, unconsumed. 3 of 7 counters have <i>zero writers</i> — leftovers of the retired usage surface
Completion bridge records	Completions visible in the record	Record readers	Working (and its failure path is tested loud)
Migration signal records	The one-way door announces itself	Future incident investigators	Just shipped (item 5); mechanism mutation-proven
Config (<code>.ideate.json</code>)	Per-project store paths/backend	Every transport open	Working
Gated: assembler prototype	The future briefing	Nobody — no handler	Deliberately dark (GP-23)
Gated: priming hook	Claim-time priming seam	Nobody — throws if enabled	Deliberately dark (GP-23); its <code>work_claims</code> counter is the only live effect

What's making it work

1. **The split by mutation semantics.** Append-only files for the record, SQLite for the board, files for steering. Each store degraded independently in the field (conway: board dead, record fine) — the architecture's central bet, paid off.
2. **Hooks as the capture floor.** Capture happens without any agent remembering: session start primes, compaction captures, commits land as records. The 635-turn session that produced stage 2 left a complete journal without a single manual "remember to log this." The *design* is what's working here; read item 6 below before trusting its current output.
3. **The mechanical-enforcement layer.** Census tests that scan the shipped tree, mutation-proven guards, cross-transport parity with a real child process. Conventions don't decay because they're executable. This is the property the eval harness must itself inherit (pre-committed thresholds are the same idea one level up).
4. **Fencing + leases on the board.** Multi-session claiming has not produced a single lost-update incident since the board shipped.
5. **GP-23's gate actually held.** Through a whole no-harness window (stages 1–2), nobody shipped intelligence machinery. The gate was mechanical (NOT_IMPLEMENTED), not a promise.

What's not working

1. **Telemetry is a write-only store.** Seven counters, four with writers, one diagnostic bin nobody calls. Data collected per its design; consumed by nothing. (Not waste — collection is cheap and the stage-3 harness may want the history — but today it answers no question.)
2. **The record's retrieval is recency + substring.** Two sessions apart, relevant context resurfaces only if it's recent or you know the scope word. The 3.1k-record store is searched far below its information content. (Parked deliberately — but it is the largest gap between "what we have" and "what we use.")
3. **The events log has no consumer-side tooling.** It answers "what happened to item X" and nothing higher-level ("what did this cycle touch," "how long do claims live"). Fine at current scale; will not scale to multi-project questions.
4. **transport/ is two layers under one name.** Working, but the next shared utility lands in a directory whose name lies about its contents.
5. **The two doors have drifted, on the door subagents are forced to use.** The shared core is genuinely shared — both adapters construct the same `WorkStateStore / WorkStateVerbs` and no board logic is written twice. But each adapter independently chooses what to wire, and two gaps are verified in the shipped tree (`04-transport-and-infra.md`): `parent_id` is absent from the CLI entirely, so a subagent cannot set or move a containment edge; and the CLI never

passes `onUnresolvedIds`, so a dangling reference id is accepted silently where MCP reports it. Both contradict `cli/ideate-work.ts`'s own header. This is the same failure *shape* as the four transport defects — a change landing on one door — except caught by structure rather than by an incident, and it is the strongest argument in this audit that the parity guard is scoped too narrowly.

6. **The capture floor is writing duplicate and false records.** Two independent defects, both found 2026-08-04 by inspecting what the phantom store had collected. First, every lifecycle hook is registered *twice* in this repository — once by the installed plugin, once by `.claude/settings.local.json` — so every captured fact is stored twice. Second, the commit-boundary hook trusts a host-side condition it never verifies and is not idempotent, so it intermittently re-reports a commit that landed long before, quoting an unrelated shell command as its cause — one commit is recorded twenty times across 38 minutes, only the first of which coincides with the commit. Together: **108 commit-boundary records in the August shard for 17 actual commits**. The ten-record digest — the always-on priming floor, and the mechanism this audit rates highest — is being filled with duplicated re-reports of old commits instead of ten distinct recent facts.

This changes what the third section of this audit can claim. “Hooks as the capture floor” is still the right design; its *current output* is not trustworthy at face value. It also matters directly for stage 3, because this repository’s record is the corpus any harness would measure. (Board items 01KZ789KW7CCP2QDQQTE4PC0MB and 01KZ78ACSCZKECN08YZVHDD3NQ.)

7. **The CLI writes to a phantom store when `cwd` isn’t the project root.** `loadConfig` does not walk up to find an enclosing project; it lazily *creates* one wherever it is pointed. Hooks defend against this explicitly; the MCP server is immune by construction; subagents are not defended at all. It already happened during this inventory — nine records landed in a store under `docs/architecture/` and are invisible to the project. Of the three findings here this is the only one that silently loses data, and it deserves a decision at the session rather than a backlog entry.

Improvement areas, ranked by leverage for the alignment session

1. **Define what the harness measures *from these areas*.** The audit table above is the candidate surface: convergence behavior (board), retention quality (record), governance adherence (steering), capture coverage (telemetry floor). GP-23’s restored instrument should gate *these*, not a re-derived recall metric.
2. **Spec the in-product signals from requirements.** If the harness needs priming-usefulness or claim-latency data, say so and wire exactly that — the three zero-writer counters are the cautionary tale of speccing signals ahead of the question.

3. **Name or split the transport layers.** Cheap: either a comment-level naming (done in 00-overview) or a directory split. Decide at the alignment session; not worth a board item alone.
4. **Events-log rollups.** A `cycle_summary`-style read over events would serve review/reporting prose that today re-derives from memory. Small, unblocked, no KG dependency.
5. **Widen the parity guard from freshness to surface.** Proposal below — the cheapest structural fix in this list, and the one with a live defect already behind it.
6. **The KG seam (post-MVP, per Dan).** When it lands, it lands behind the restored instrument: semantic seed + PPR assembly, with the record's typed references as its first edges. The audit's item 2 is its justification, quantified by the harness it must ship behind.

Proposal: one core, mechanically-thin doors

The instinct to want “a singular chokepoint, easy to test and abstract” is right, and it is *almost* satisfied already — but the goal cannot be one door. Hooks are processes and subagents have no MCP tools; the CLI is structural, not a shortcut. The achievable version is **one core with doors whose thinness is checked rather than trusted**.

Three checks, in increasing cost:

1. **Surface-parity census** (cheap, mechanical, fits the existing enforcement layer). A test that derives the MCP verb surface and the CLI subcommand surface *by scanning the shipped tree* — the same technique the prose-census tests already use — and asserts every verb exists on both doors with the same argument set, or carries a named, reasoned exemption (`sweep` is legitimately CLI-only; `steering_*` is legitimately MCP-only). Both drifts above fail this test on day one.
2. **Behavioral parity for the board**, extending `transport-parity.test.ts` from the record to the board: for each verb, the same inputs through either door produce the same item state and the same error taxonomy. This is what would have caught `onUnresolvedIds`, which a surface census might miss because it is a return-value gap rather than a missing flag.
3. **A structural thinness rule** (optional, decide at the session): CLI and MCP handlers may parse, serialize, and call verbs — nothing else. Today's handlers already honor this; the rule would keep the next one honest and make “where does logic live” un-negotiable rather than conventional.

Checks 1 and 2 are small, unblocked, and need no KG or harness dependency. The phantom-store finding is not a parity problem and none of the three checks would catch it; it wants its own fix — project-root discovery by walk-up, with lazy creation reserved for a true root — and it is the one item here that is losing data today.

The open question for the session is whether the two surface drifts get fixed as defects now, or deliberately ratified as intended asymmetries — the CLI omitting `parent_id` may have been a decision nobody wrote down, and the answer changes whether `ideate:decomposer` can ever

produce containment edges from a subagent.

What this audit deliberately did NOT do

Measure anything. Every verdict above is structural (who imports what, who calls what) or incident-grounded (conway, the 81KB overflow, the telemetry census). Quantifying “is the record’s content *good*” is the harness’s job — and why the diagnosis of the old harness (next step) matters before any design.